

Tabla comparativa de métodos de ordenamiento

Se comparan cinco algoritmos (burbuja, quicksort, shellsort, mezcla directa y mezcla natural) con criterios técnicos de complejidad, eficiencia práctica, estabilidad, tipo de ordenamiento y aplicaciones. Las justificaciones se enfocan en el comportamiento real del algoritmo y su costo computacional.

Método	Complejidad (Mejor / Promedio / Peor)	Eficiencia y justificación	Estabilidad	Tipo (interno/externo)	Aplicaciones comunes
Burbuja	$O(n)$ / $O(n^2)$ / $O(n^2)$	Baja eficiencia para volúmenes medianos o grandes; requiere múltiples pasadas y comparaciones.	Estable; requiere múltiples pasadas y comparaciones.	Interno	Algoritmo educativo, listas pequeñas.
Quicksort	$O(n \log n)$ / $O(n \log n)$ / $O(n^2)$	Muy eficiente en práctica por particionado y buen caso promedio.	Generalmente inestable; puede requerir optimizaciones como el uso de caché.	Interno	Algoritmo estándar en bibliotecas de programación.
Shellsort	$\approx O(n \log n)$ / depende de secuencia de incrementos	Eficiencia intermedia: mejora sobre inserción al reducir desplazamientos largos mediante saltos.	Generalmente inestable.	Interno	Algoritmo educativo, listas pequeñas.
Mezcla directa (merge sort)	$O(n \log n)$ / $O(n \log n)$ / $O(n \log n)$	Eficiencia estable y predecible; divide y combina.	Estable; costo regular.	Interno y externo	Ordenación de registros, procesamiento de datos.
Mezcla natural	$O(n)$ / $O(n \log n)$ / $O(n \log n)$	Aprovecha "runs" ya ordenadas en la entrada; puede reducir pasadas.	Estable; puede reducir pasadas.	Interno y externo	Clasificación de archivos de secuencia.

Conclusión técnica: quicksort suele dominar en ordenamiento interno general por su alto rendimiento promedio; merge sort (mezcla directa/natural) destaca cuando se requiere estabilidad y manejo de grandes volúmenes, especialmente en contextos externos; burbuja y shellsort se reservan para escenarios educativos o de menor escala.